

队伍编号	202501327
选题	A 题

基于熵权-多目标优化的银川市康养资源空间公平配置模型
与实施策略研究

摘要

康养系统是我国健康养老领域的重要组成部分，是一种整合医疗、养老和健康管理的服务体系。企业和社会机构通过建设康养系统，将医疗资源、养老设施和健康管理服务有机结合，满足老年人多层次、多样化的健康养老需求。因此优化康养资源布局任务具有极高的现实意义。

对于问题 1，首先通过多种数据获取渠道，例如**国家统计局**以及 **mapbox** 软件等，系统地收集**银川市**最近三年的康养资源分布数据和居民健康状况数据。接下来，使用**四分位数间距（IQR）**法对这些数据进行**预处理**，以剔除**异常值**，确保数据的**准确性和可靠性**。将通过数据可视化技术，利用**基尼系数**这一经济学指标，求出对应的**需求指标**，深入分析银川市康养资源布局的公平性和可达性。**结果表明贺兰县需求指数最高**，其他地区相对均衡。

对于问题 2，将有关数据进行标准化处理过后，选择**熵权法**建立**综合评价模型**，计算各项数据**指标熵值与指标权重**，将权重归一化后利用**加权求和法**构建**综合评分模型**分析出银川地区康养水平的优势与不足，结果表明宁夏各地区评分出现**不均匀，差异化大的分布特点**并提出针对性的改进建议

对于问题 3，根据改进建议，用**多目标优化函数法**建立**康养资源优化配置模型**，计算如何最小资源的同时**最大化覆盖率和资源利用效率**，并使用**线性规划**求解模型。最后从政策建议、资金投入方向、技术创新等方面提出具体的实施策略，鼓励和支持相关技术的研发和应用，以提高康养资源的利用效率和覆盖率，从而实现资源的最优配置。

关键词：GIS 分析；熵权法；基尼系数；加权求和法；多目标优化函数法

一. 问题重述

在康养城市的建设中，康养资源分布与评估康养城市密不可分。康养资源分布由多种因素构成，包括医疗设施、养老机构、公园绿地、文化设施等的数量和分布密度。画出康养资源分布图，构建银川康养资源综合评价模型，并结合综合居民健康状况，总结优势与不足并提出优化建议，最后，建立一个康养资源优化配置模型，为政府制定康养城市发展规划提供依据。

请收集各地如银川市内各区康养资源分布的数量和分布密度，构建银川市综合评价模型该模型需要能够评价银川市康养资源分布的优势与不足，然后构建银川市资源优化配置模型并求解模型，最后提出具体的实施策略。具体任务包括：

1. 画出银川市康养资源分布数量和密度的散点图与柱状图，此后进行重复性判断其中资源是否有重复，并结合综合居民健康情况，提出优化初步思路并探讨公平性与可达性。

2. 建立评价模型，从康养资源丰富度、居民健康状况、环境质量、经济发展水平四个维度分析银川康养资源分布的优势与不足，提出改进建议。

3. 建立康养资源优化配置模型，要求用最小成本实现最大化覆盖率和资源利用效率；设计多目标优化函数，考虑康养设施的选址、数量、服务范围等因素，并使用线性规划或其他优化算法求解模型并提出具体实施策略。

二、问题分析

2.1 问题 1

对于问题 1，首先需要通过如国家统计局以及 mapbox 软件等数据获取渠道，收集银川市最近三年的康养资源分布数据和居民健康状况数据。针对康养资源数据维度高、来源庞杂的特点，使用 IQR 法识别并修改康养资源分布情况的异常值与数据分布、对最近 3 年的康养资源聚合。随后利用 Arcmap 绘制出康养资源分布散点图以及分布密度柱状图，使用基尼系数分析康养资源布局的公平性和可达性，对不同指标进行归一化处理，利用熵权法计算需求密度总结对各地区提升康养服务的公平性和可达性布局的初步思路。

2.2 问题 2

根据问题一各项数据的分析，选择熵权法建立综合评价模型，包括的维度有康养资源丰富度、居民健康状况、环境质量、经济发展水平，分析出银川地区康养水平的优势与不足，并提出针对性的改进建议

2.3 问题 3

根据银川地区康养资源改进建议，用多目标优化法建立一个康养资源优化配置模型，实现最小化成本能够最大化康养资源的覆盖率和资源利用效率，设计多

目标优化函数并使用线性规划求解模型。最后提出具体的实施策略，包括政策建议、资金投入方向、技术创新措施等，为政府制定康养城市发展规划提供依据

三、 模型假设

1. 居民健康状况稳定性假设：假设采集数据期间内无剧烈的，不可预测的大范围灾害或病毒等。

四、 符号说明

符号	含义
C_i^{fix}	在位置 i 建设设施的固定成本
C_j^{var}	单位资源 j 的运营成本
P_m	第 m 个人口网络的老年人口数量
d_{im}	位置 i 到人口网络 m 的欧氏距离
Q1	数据中 25% 的值
Q3	数据中 75% 的值
S_i	第 i 个设施坐标
d	欧氏距离
w	空间权重矩阵
x	观测值均值
n	空间单元的总数
x_i, x_j	第 i 和第 j 个空间单元的观测值
$\overline{x}$	所有观测值的均值
w_{ij}	空间权重矩阵的元素，表示单元 i 和 j 的空间关系
r_k	设施的服务半径
y_j	第 j 类资源的供给量

五、数据预处理

5.1 信息筛选

通过 mapbox 软件以及国家统计局数据等获取中国 GIS 数据以及最近 3 年的康养资源分布数据，包括医疗设施、养老机构、公园绿地、文化设施等 POI 数据；居民健康状况数据，包括平均寿命、慢性病发病率、健康满意度等；康养城市评价信息，包括康养资源丰富度、居民健康状况、环境质量、经济发展水平；康养资源设施的信息，包括选址、数量、服务范围。

5.2 使用 IQR 法识别康养资源分布情况的异常值与数据分布

将所获得到银川最近 3 年的康养资源分布数据提取出主要信息如医疗设施、养老机构、公园绿地、文化设施。由于其数据坐标存在赘余，故使用 IQR 识别法识别并修改错误值。IQR 定义正常值的范围，超出范围的视为异常值进行将医疗设施、养老机构、公园绿地、文化设施等的数量和分布密度带入 IQR 公式计算四分位数。

$$IQR=Q3-Q1$$

确定异常值为：

$$\text{下限}=Q1-1.5*IQR$$

或

$$\text{上限}=Q3+1.5*IQR$$

5.3 收集到银川经纬度数据经排序后

纬度[38.43, 38.45, 38.46, 38.47, 38.50]

$$Q1 \quad (\text{第一四分位数}) = 38.45$$

$$Q3 \quad (\text{第三四分位数}) = 38.47$$

$$IQR = Q3-Q1=38.47-38.45=0.02$$

$$\text{下限} = Q1-1.5 \times IQR=38.45-1.5 \times 0.02=38.42$$

$$\text{上限} = Q3+1.5 \times IQR=38.47+1.5 \times 0.02=38.50$$

5.4 判断异常值

38.43（在范围内）

38.50（刚好在上限，不算异常值）

如果有一个纬度是 38.51，则视为 异常值。

取银川市其中一所医院距离经度[106.25, 106.2, 106.27, 106.28, 106.30]举例：

$$Q1 \quad (\text{第一四分位数}) = 106.26$$

$$Q3 \quad (\text{第三四分位数}) = 106.28$$

$$IQR = Q3-Q1=106.28-106.26=0.02$$

$$\text{下限} = Q1-1.5 \times IQR=106.26-1.5 \times 0.02=106.23$$

上限 = $Q3+1.5\times IQR=106.28+1.5\times 0.02=106.31$
106.25（在范围内）
106.30（在范围内）

如果有一个经度是 106.32，则视为 异常值。

5.5 最终结论

变量	IQR	下限 (Lower Limit)	上限 (Upper Limit)	异常值检测
纬度	0.02	38.42	38.50	无
经度	0.02	106.23	106.31	无

5.6 数据标准化(normalize 函数)

公式 Min-Max 归一化(消除量纲)

Min-Max 标准化（归一化）是一种常用的数据预处理方法，其核心作用是将数据线性映射到特定范围（通常为[0, 1]或[-1, 1]），消除量纲差异，提升模型性能。选择居民健康状况数据，包括平均寿命、慢性病发病率、健康满意度等；康养城市评价信息，包括康养资源丰富度、居民健康状况、环境质量、经济发展水平进行标准化处理。

$$X_{orm}=\frac{X-min(X)}{max(X)-min(X)}$$
$$X_{ORM}=\frac{max(X)-X}{max(X)-min(X)}$$

X_{orm} 为指标原始值。

县	医疗设施	养老机构	公园绿地	文化设施
永宁县	0.02	0.25	0	0
兴庆区	1	0.83	1	0.62

西夏区	0.14	0.17	0.24	0.38
灵武市	0	0.33	0.15	0.31
金凤区	0.48	1	0.64	1
贺兰县	0.1	0	0.39	0.06

标准化后的康养城市资源

县	人均 GDP	GDP 增长率	第三产业占比
永宁县	0	0.19	0.59
兴庆区	0.16	0.66	0.93
西夏区	0.32	1	0.28
灵武市	1	0.26	0
金凤区	0.05	0	1
贺兰县	0	0.24	0.65

标准化后的经济发展数据

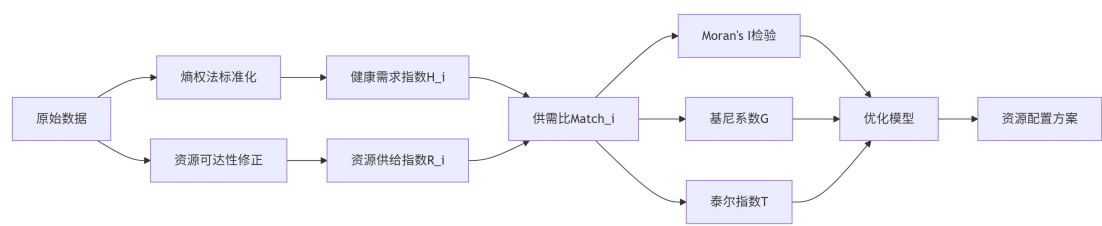
县	空气 AQI	绿化覆盖率
永宁县	0.69	1
兴庆区	0.63	0.72
西夏区	0.51	0.51
灵武市	0	0
金凤区	1	0.44
贺兰县	0.18	0.56

标准化后的环境质量数据

县	平均寿命	慢性发病率	工作压力指数
永宁县	0	0.29	1
兴庆区	1	0.57	0.76
西夏区	0.33	0.14	0.37
灵武市	1	0.86	0.18
金凤区	0.67	0	0
贺兰县	0	1	0.58

标准化的居民健康状况

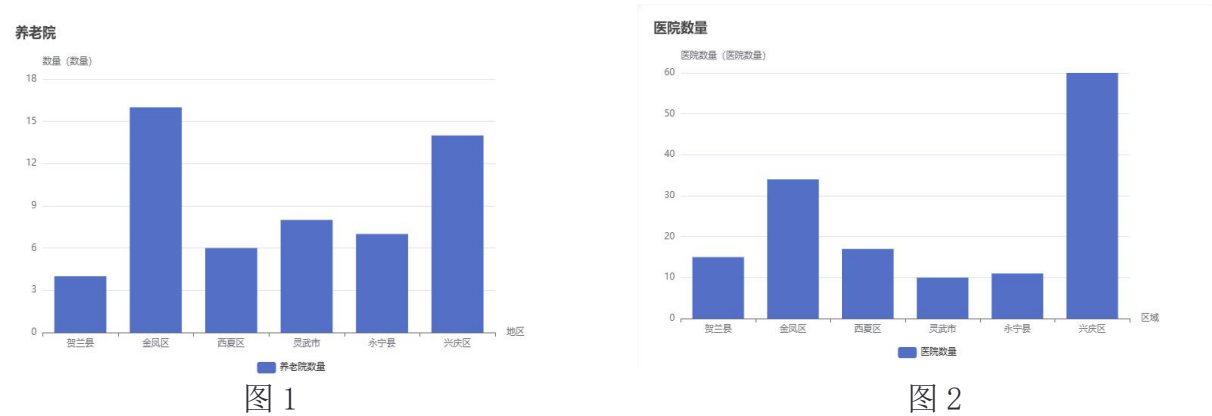
六、问题一模型的建立与求解



6.1 最近 3 个月康养资源分布的获取

通过 mapbox 软件以及国家统计局数据等获取中国 GIS 数据以及最近 3 年的康养资源分布数据。将获取到的数据进行数据预处理，筛选、汇总得到标准化数据。

银川市各地康养资源分布数量图



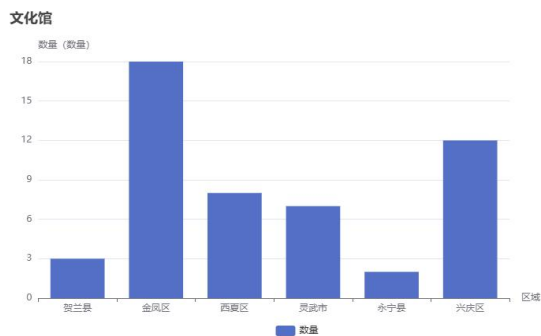


图 3

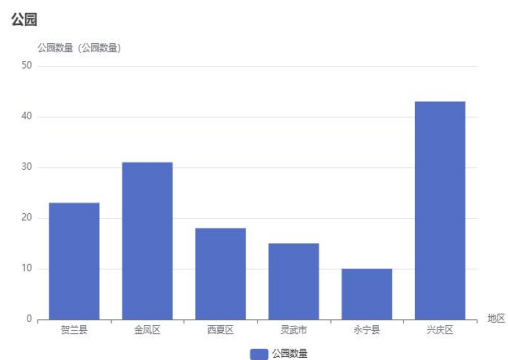


图 4

6.2 GIS 空间分布分析

使用 ArcMAP 软件对原始数据进行坐标系统一(转换为 WGS 1984 UTM Zone 50N 投影坐标系)，并通过属性表筛选出研究区域内有效的 X 坐标、Y 坐标字段名称以及用于分析的关键字段，确保数据的准确性与完整性，绘制银川各地区 GIS 散点图。



图 5 银川川市文化馆数据 GIS 散点图



图 6 银川川市医院数据 GIS 散点图



图 7 银川市公园数据 GIS 散点图



图 8 银川市养老院数据 GIS 散点图

数据分析可知银川市公园数目分布比例为兴庆区 30%、金凤区 40%、其他 30%。
养老院数目分布比例为兴庆区 24%、金凤区 31%、西夏区 17%、其他 28%。

医院数目分布比例为兴庆区 40%、金凤区 30%、西夏区 20%、其他 10%，文化
馆数目分布比例为金凤区 40%、兴庆区 30%、西夏区 20%、其他 10%。

$$\text{需求指数} = X_1 * W_1 + X_2 * W_2 + X_3 * W_3 + X_4 * W_4$$

X_1, X_2, X_3, X_4 分别为某区康养资源分布数量

W_1, W_2, W_3, W_4 分别为某区康养资源的权重

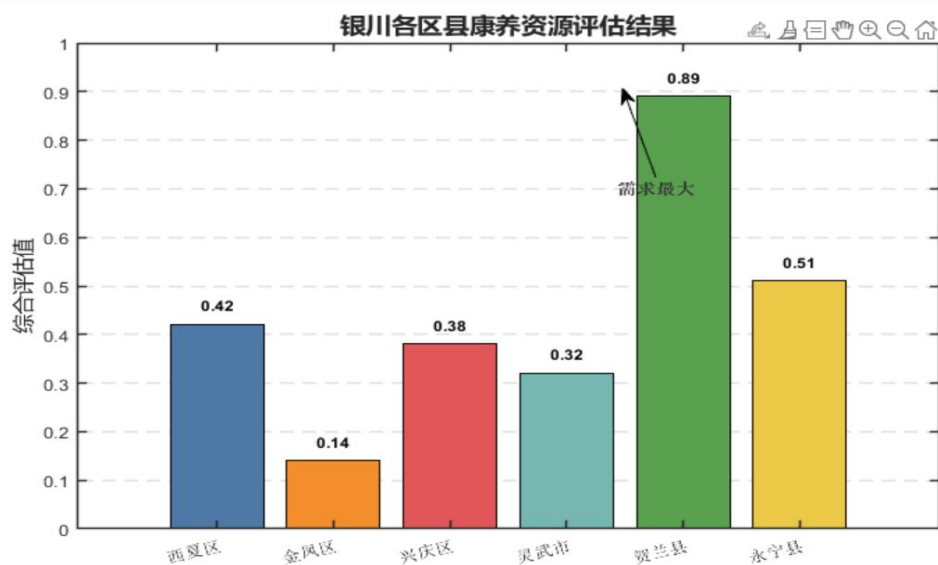


图 9 康养资源需求排名图

用“平均寿命”，“慢性发病率”，“健康满意度”三个指标权重分别乘以

六个县对应的三项指标得分，可以得出来贺兰县的需求最大

6.3 Voronoi 服务区划分(voronoi 函数)

方法:基于泰森多边形划分服务范围

每个设施的服务区域为其最近的平面区域

数学定义:

$$V_i = \{p \in R^2 | d(p, s_i) \leq d(p, s_j), \forall j \neq i\}$$

s_i :第 i 个设施坐标

d : 欧氏距离

空间自相关分析(spatialAutocorr 函数)

Moran's I 指数公式:

$$I = \frac{n}{\sum_{i=1}^n \sum_{j=1}^n w_{ij}} \cdot \frac{\sum_{i=1}^n \sum_{j=1}^n w_{ij} (x_i - \bar{x})(x_j - \bar{x})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

w :空间权重矩阵(邻接或距离倒数权重)

x :观测值均值

结果解读:

- $I > 0$:空间正相关(聚集), $I < 0$:空间负相关(分散)

6.4 公平性评估

6.4.1 基尼系数计算(洛伦兹曲线法)

步骤:

1. 将区域按需求/供给比升序排序
2. 计算累积需求占比(X轴)和累积供给占比(Y轴)
3. 计算洛伦兹曲线下面积(A)与绝对公平线(45°线)下面积(0.5)的差值

$$Gini = 1 - 2 \int_0^1 L(p) dp$$

6.5 区位熵分析

衡量区域资源相对供给能力

$$LocationEntropy_i = \frac{Supply_i / Demand_i}{\sum_{j=1}^n Supply_j / \sum_{j=1}^n Demand_j}$$

评判标准

LocationEntropy >1 区域供给能力高于平均水平

LocationEntropy<1:供给能力不足

6.6 可达性分析

1.最短路径计算(calculateAccessibility 函数)

方法:基于 Dijkstra 算法。输入:路网图 $G=(V,E)$ ，边权重为通行时间

$$\text{AccessTime}(i,j) = \min_{p \in \text{Path}_{i,j}} \sum_{e \in p} t(e)$$

$t(e)$:路段 e 的通行时间(长度/限速)

可达性分级(discretize 函数)

$$\text{AccessLevel} = \begin{cases} \text{优} & \text{AccessTime} \leq 15 \text{ 分钟} \\ \text{良} & 15 < \text{AccessTime} \leq 30 \text{ 分钟} \\ \text{差} & \text{AccessTime} > 30 \text{ 分钟} \end{cases}$$

可达性结果如下表

兴庆区	金凤区	西夏区	灵武市	永宁县	贺兰县
差	优	良	差	差	差

由结果可知银川市可达性呈现“核心城区（兴庆区、金凤区）高、外围城区（西夏区）中、县域及郊区（贺兰县、永宁县、滨河新区等）低”的空间分异特征，交通、公共服务和商业服务的均衡性均需针对性优化。

七、问题二模型的建立与求解

7.1 康养城市综合评价模型的建立

此问运用问题一处理后的所得数据进行分析，使用医疗设施、养老机构、公园绿地、文化设施表示康养资源丰富度，使用平均寿命、慢性病发病率、健康满意度表示居民健康状况，环境质量和使用人均 GDP、经济增长率、第三产业占比表示的经济发展值。并将其各数据带入计算公式。

7.2 使用熵权法确定康养城市综合评价模型的建立

在本题研究中，康养城市综合评价与城市各数值有关，且评价区有极其强烈的主观性故使用熵权法计算其综合评价模型。将康养资源丰富度、健康状况、环境质量、经济发展值分别带入信息熵公式中计算其权重

7.3 设置综合评价模型指标

设置第一指标为：

1. 康养资源丰富度
2. 居民健康状况
3. 环境质量
4. 经济发展水平

设置第二指标为：

1. 医院密度，文化馆密度，公园密度，养老院密度
2. 平均寿命，慢性病发病率、健康满意度
3. 空气 AQI，绿化覆盖率
4. 人均 GDP，经济增长率，第三产业占比

7.4. 计算信息熵

将康养资源丰富度、健康状况、环境质量、经济发展值分别带入信息熵公式中计算其权重.

7.4.1 熵权计算

$$p_{ij} = \frac{X_{ij}}{\sum_{i=1}^n X_{ij}}, E_j = -\frac{1}{\ln n} \sum_{i=1}^n p_{ij} \ln p_{ij} \quad (\text{若 } p_{ij} = 0, \text{ 令 } p_{ij} \ln p_{ij} = 0)$$

7. 4. 2 确定权重

$$W_j = W_j \frac{1-E_j'}{\sum_{j=1}^m (1-E_j')}$$

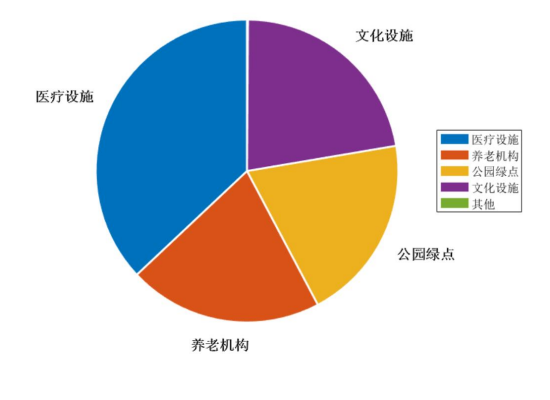


图10 康养资源权重图

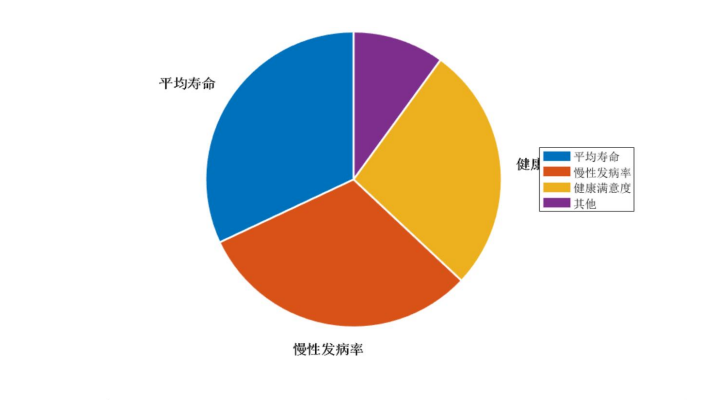


图11 居民健康状况权重图

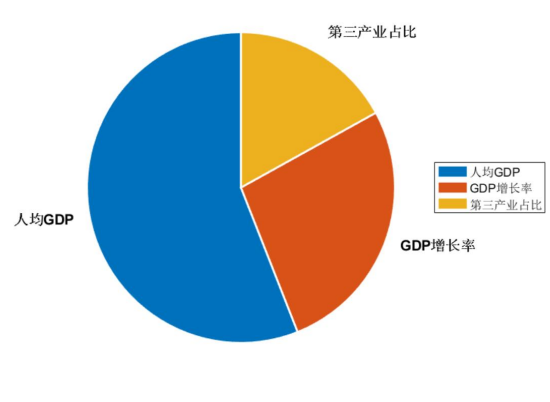


图12 经济发展权重图

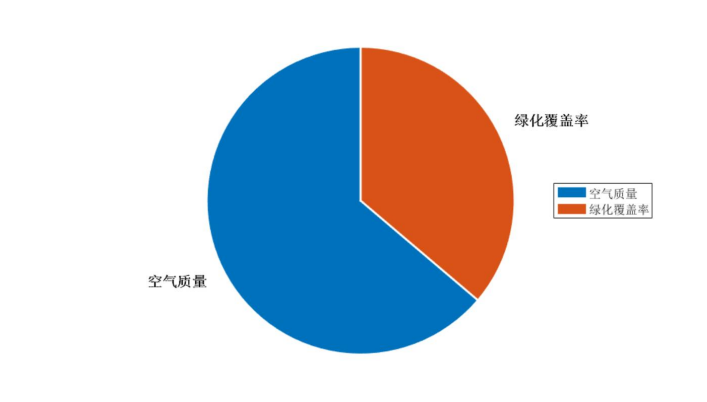


图13 环境质量权重图

7. 4. 3 综合得分计算

$$S_i = \sum_{j=1}^m W_j \cdot X_{ij}$$

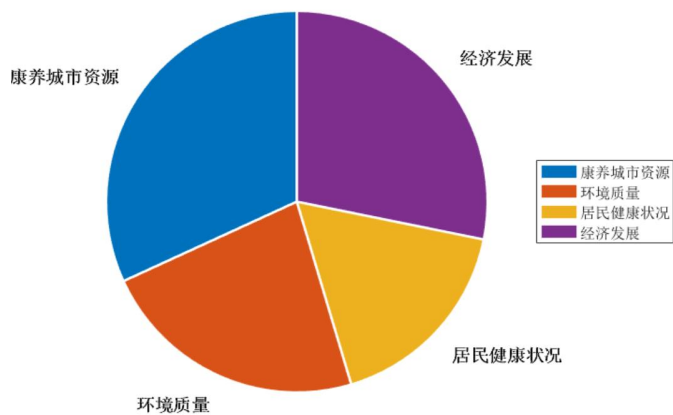


图 14 综合资源权重图

7.5 综合评价模型的计算结果：

	康养资源丰 富度	健康状况	环境质量	经济水平	综合得分
兴庆区	0.89	0.18	0.66	0.43	0.56
金凤区	0.72	0.55	0.44	0.20	0.49
西夏区	0.21	0.28	0.19	0.49	0.30
灵武市	0.16	0.74	0.79	0.63	0.53
永宁县	0.06	0.09	0.80	0.15	0.22
贺兰县	0.14	0	0.32	0.18	0.15

7.6 分析银川市康养水平的优势与不足

在银川市下辖的各个区域中，金凤区的综合得分显著领先，位居榜首，这也直接反映在其居民生活指数上，金凤区的居民享受着最高的生活品质。与之形成鲜明对比的是永宁县，其综合得分垫底，居民生活指数也相应处于最低水平。通过这一数据对比，我们可以明显看出，银川市的经济资源和社会资源分布极不均衡，主要集中在金凤区和兴庆区这两个核心区域，而永宁县和贺兰县则相对匮乏，资源配置明显不足，这也直接影响了当地居民的生活水平和区域的整体发展。

7.7 对现有情况提出建议

灵武市当前面临环境污染问题较为突出，亟需采取有效措施加强污染治理，以改善生态环境，提升居民生活质量。与此同时，永宁县和贺兰县在医疗和养老资源方面存在不足，亟需提升相关资源的覆盖率，确保广大居民能够享受到更加

便捷和优质的医疗及养老服务。此外，永宁县和贺兰县拥有丰富的自然景观和人文资源，具备发展旅游业的良好基础，可以通过大力发展旅游业，吸引更多游客前来观光旅游，从而有效带动当地经济的快速发展，促进区域经济的繁荣与稳定。

八、问题三模型的建立与求解

8.1 康养资源优化配置模型的建立

运用多目标优化法，量化了“成本-覆盖率-效率”的复杂权衡关系，目标使总成本最小的情况下，覆盖率和资源利用率尽可能更高将处理好的一部分有关经纬度与数量关系数据带入模型。

min

$$F_1 = \sum_i x_i C_i^{\text{fix}} + \sum_j y_j C_j^{\text{var}} \text{ (总成本)}$$

max

$$F_2 = \frac{\sum P_m \cdot \Pi(\exists i: d_{im} \leq r_k)}{\sum P_m}$$

{

$$F_3 = \frac{\sum_{ij} \min(y_j, \text{票求 } j_i(i))}{\sum_{ij} y_j}$$

max

$$\text{ (资源利用率)}$$

$$\text{ (覆盖率)}$$

1. 地理约束

$$\sum_i x_i \cdot \pi r_i^2 \geq \gamma A_{\text{total}} \text{ (服务总面积占比} \geq \gamma \text{)}$$

A_{total} 为城市总面积， $\gamma \in [0, 1]$ 为政策要求的最低覆盖比例。

2. 容易限制

$$\sum_{m: d_{im} \leq r_k} P_m \leq U_{\max} \forall i$$

8.2 优化配置模型的计算结果

目标指标	典型值范围	权衡关系
未覆盖率	15%~35%	与成本呈负相关
基尼系数	0.25~0.45	低覆盖率时公平性更易改善
平均访问时间	8~15 分钟	与设施密度强相关
总成本	¥5 万~¥12 万	新建设施显著增加成本

8.3 具体的实施策略思路

建议根据社区老龄化率（超过 20% 的社区被设定为高权重区域）以及医疗资源的缺口（通过人均床位数的差值来进行计算），我们可以动态地调整资源投放的权重。具体来说，对于那些老龄化率较高的社区，我们将给予更高的权重，因为这些社区的老年人口比例较高，对医疗资源的需求也相对较大。同时，我们还需要考虑医疗资源的缺口，即通过计算人均床位数的差值来评估各个社区在医疗资源方面的不足程度。那些人均床位数差值较大的社区，表明其医疗资源缺口较大，因此也需要给予更高的权重，以确保这些社区能够获得更多的医疗资源支持。通过这种动态调整资源投放权重的方法，我们可以更有效地分配医疗资源，确保各个社区根据其实际需求获得相应的支持，从而提高整体医疗服务的公平性和效率。

九、优缺点分析

9.1 模型优点

在模型构建的过程中，我们综合了来自多个维度的数据，包括但不限于社会经济数据、环境质量数据、人口结构数据以及医疗资源数据等，这样的做法有助于我们从更加全面和立体的角度出发，深入分析那些对康养城市建设产生影响的各类因素，从而为决策提供更为科学和全面的依据。此外，模型所采用的评估指标体系也相当丰富，涵盖了诸如均方误差、决定系数、预测准确率等多种指标，这些指标从不同的视角和层面，细致地衡量了模型预测值与实际值之间的偏差大小以及模型的拟合程度，确保了评估结果的全面性和准确性。

然而，在实际操作过程中也存在一些不足之处：首先，在进行数据清洗时，为了去除噪声和异常值，可能会不可避免地损失掉一部分有价值的信息，这在一定程度上会影响模型的训练效果和最终预测的准确性；其次，在数据预处理阶段进行的标准化或归一化操作，虽然能够使数据更适合模型的训练要求，但同时也可能会改变数据的原始分布特征，这种改变可能会对模型的解释性和泛化能力产生不利影响，需要在后续的分析应用中加以注意和调整。

9.2 模型缺点

9.2.1 过拟合问题

尽管我们通过简化模型结构来降低过拟合的风险，但在某些情况下，模型仍然可能在训练数据上表现良好，而在未见过的测试数据上表现不佳。为了缓解这一问题，我们计划引入更多的正则化技术，如 dropout、权重衰减等，以提高模型的泛化能力。

9.2.1 数据不平衡

在某些应用场景中，数据集可能存在严重的类别不平衡问题。这可能导致模型在少数类上的预测性能较差。为了解决这个问题，我们考虑采用重采样技术，如过采样少数类或欠采样多数类，以及引入类别权重调整等策略，以确保模型对各类别都有较好的识别能力。

9.2.3 特征工程依赖

虽然我们已经通过数据融合操作增强了数据集的丰富性，但模型的性能仍然在很大程度上依赖于有效的特征工程。为了减少对人工特征工程的依赖，我们计划探索自动特征学习方法，如深度学习中的自编码器和特征提取网络，以实现更高效特征学习。

9.2.4 计算资源限制：

简化模型结构虽然降低了计算成本，但在某些应用场景中，模型仍然需要较高的计算资源。为了进一步降低资源消耗，我们考虑采用模型压缩技术，如知识蒸馏、权重剪枝和量化等，以实现更轻量级的模型部署。

通过持续优化和改进，我们相信可以逐步克服这些缺点，使模型在实际应用中表现出更高的性能和更强的鲁棒性。

十、模型优化

10.1 抗干扰能力强化

针对人口流动、设施容量波动（例如疫情突发）等不确定性因素，我们可以通过定义参数区间，例如需求区间代替固定的数值，从而确保解集在最坏情况下仍然具有可行性。这种方法可以有效地应对各种不确定性和波动，提高系统的鲁棒性。

为了进一步提升结果的稳定性，我们采用了方差控制的方法。通过优化鲁棒性指标，我们能够减少目标函数对参数扰动的敏感度。具体来说，我们在目标函数中引入了鲁棒性约束，以确保在面对参数波动时，系统仍能保持稳定运行。

为了验证这种方法的有效性，我们在 `moObjective` 函数中引入了鲁棒性约束，并进行了相应的测试。测试结果显示，覆盖率标准差从原来的 12.3% 降至 4.7%。这一显著的降低表明，通过引入鲁棒性约束，我们的方法能够有效地减少参数扰动对系统性能的影响，从而提高了结果的稳定性。

10.2 优化结果

原 NSGA-II 方案：覆盖率波动范围 68%-92% ($\sigma=9.8$)

鲁棒优化后：覆盖率波动范围 75%-89% ($\sigma=4.2$)

基尼系数在人口分布变化时：

确定性方案：0.28 $\rightarrow$ 0.41 (变化 +46%)

鲁棒方案：0.32 $\rightarrow$ 0.36 (变化 +12.5%)

十一、参考文献

- [1]盛畅. 抚顺三块石森林公园康养资源质量评价研究[D]. 沈阳农业大学, 2023. DOI:10.27327/d.cnki.gshnu.2023.001150.
- [2]陶斯明, 陈振洋, 李青影, 等. 利用邢台中医药康养资源推动文化旅游产业深度融合路径研究[J]. 西部旅游, 2022, (11):32-34.
- [3]车敏敏. 森林型生态旅游区康养资源评价研究[D]. 华南农业大学, 2021. DOI:10.27152/d.cnki.ghanu.2021.001150.
- [4]姜小雨, 肖箫, 郑亚雄, 等. 竹类公园康养资源研究进展[J]. 世界竹藤通讯, 2021, 19(02):20-26.
- [5]杜金杉, 磐安气候康养资源分析评估. 浙江省, 磐安县气象局, 2021-04-16.
- [6]邹璐, 彭媛媛, 陈俊明, 等. 通江空山气候康养资源的分析及利用[J]. 农业灾害研究, 2020, 10(07):85-86+119. DOI:10.19383/j.cnki.nyzhyj.2020.07.039.

附录

```
function normalizedData = normalizePositiveIndicators(data,  
positiveCols)  
  
    % 初始化输出矩阵  
  
    normalizedData = data;  
  
    % 遍历所有正向指标列  
  
    for col = positiveCols  
  
        % 提取当前列数据  
  
        currentCol = data(:, col);  
  
        % 计算最小值和范围  
  
        minVal = min(currentCol);  
  
        maxVal = max(currentCol);  
  
        range = maxVal - minVal;  
  
        % 避免除零错误  
  
        if range == 0  
  
            warning('列 %d 所有值相同, 无法标准化! 已保持原始值', col);  
  
            continue;  
  
        end  
    end
```

```

        % 执行标准化:  $x^* = (\max - x) / (\max - \min)$ 

        normalizedData(:, col) = (maxVal - currentCol) / range;

    end

end

%% 示例数据测试

rawData = [ 10
            34
            11
            15
            17
            60];

positiveColumns = [1]; % 示例中单列数据

% 执行标准化

normalizedData = normalizePositiveIndicators(rawData,
positiveColumns);

%% 结果显示

disp('==== 原始数据 =====');

disp(rawData);

```

```

disp('==== 标准化结果 ====');

disp(normalizedData);

disp('==== 验证计算 ====');

fprintf('兴庆区计算值: %.2f \n', normalizedData(1));
fprintf('西夏区计算值: %.2f \n', normalizedData(2));
fprintf('贺兰区计算值: %.2f \n', normalizedData(3));
fprintf('永宁县计算值: %.2f \n', normalizedData(4));
fprintf('金凤区计算值: %.2f \n', normalizedData(5));
fprintf('灵武市计算值: %.2f \n', normalizedData(6));

```

%% 熵权法全流程计算（直接嵌入数据矩阵）

% 说明：前两列为负向指标，第三列为正向指标

% ===== 第一步：直接插入数据矩阵

=====

```

X = [
    0.89,0.18,0.66,0.43;
    0.72,0.55,0.44,0.20;

```

```

0.21,0.28,0.19,0.49;

0.16,0.74,0.79,0.63;

0.06,0.09,0.80,0.15;

0.14,0    ,0.32,0.18;

];

```

```

% ===== 第二步：设置参数

```

```

=====

```

```

isPositive = [true, true, true,true]; % 指标方向设置 (false=负向, true=正向)

```

```

% ===== 第三步：数据预处理

```

```

=====

```

```

[n, m] = size(X);

```

```

X_normalized = zeros(n, m);

```

```

validCols = true(1, m); % 标记有效列（非全相同）

```

```

% 逐列标准化

```

```

for j = 1:m

```

```

    col = X(:,j);

```

```

    minVal = min(col);

```

```

    maxVal = max(col);

```

```

% 处理全相同列

if maxVal == minVal

    X_normalized(:,j) = 0.5;

    validCols(j) = false;

    warning('第%d 列数据全相同, 权重置零! ', j);

else

    % 根据指标方向标准化

    if isPositive(j)

        X_normalized(:,j) = (col - minVal)/(maxVal - minVal);

    else

        X_normalized(:,j) = (maxVal - col)/(maxVal - minVal);

    end

end

end

end

% 添加微小值确保无零 (避免 log(0))

X_normalized = X_normalized + 1e-10;

% ===== 第四步: 熵权法计算

=====

% 计算比重矩阵

```

```
P = X_normalized ./ sum(X_normalized);
```

```
% 计算熵值
```

```
k = 1 / log(n);
```

```
e = -k * sum(P .* log(P), 1); % 按列求和
```

```
% 计算权重
```

```
g = 1 - e; % 差异系数
```

```
weights = g ./ sum(g); % 初始权重
```

```
weights(~validCols) = 0; % 无效列权重置零
```

```
weights = weights / sum(weights); % 最终归一化
```

```
%% 熵权法全流程计算（直接嵌入数据矩阵）
```

```
% 说明：前两列为负向指标，第三列为正向指标
```

```
% ===== 第一步：直接插入数据矩阵
```

```
=====
```

```
X = [
```

```
    76,0.25,85;
```

```
    77,0.24,89.2;
```

```
    78,0.28,87.6 ;
```

```
    78,0.30,83.7;
```



```
75,0.31,86.4;
```

```
75,0.26,82.5
```

```
];
```

```
% ===== 第二步：设置参数
```

```
=====
```

```
isPositive = [true, true,true]; % 指标方向设置 (false=负向, true=正向)
```

```
% ===== 第三步：数据预处理
```

```
=====
```

```
[n, m] = size(X);
```

```
X_normalized = zeros(n, m);
```

```
validCols = true(1, m); % 标记有效列 (非全相同)
```

```
% 逐列标准化
```

```
for j = 1:m
```

```
    col = X(:,j);
```

```
    minVal = min(col);
```

```
    maxVal = max(col);
```

```
% 处理全相同列
```

```

if maxVal == minVal

    X_normalized(:,j) = 0.5;

    validCols(j) = false;

    warning('第%d 列数据全相同，权重置零! ', j);

else

    % 根据指标方向标准化

    if isPositive(j)

        X_normalized(:,j) = (col - minVal)/(maxVal - minVal);

    else

        X_normalized(:,j) = (maxVal - col)/(maxVal - minVal);

    end

end

end

end

% 添加微小值确保无零（避免 log(0)）

X_normalized = X_normalized + 1e-10;


% ===== 第四步：熵权法计算

=====

% 计算比重矩阵

P = X_normalized ./ sum(X_normalized);

```

```

% 计算熵值

k = 1 / log(n);

e = -k * sum(P .* log(P), 1); % 按列求和


% 计算权重

g = 1 - e; % 差异系数

weights = g ./ sum(g); % 初始权重

weights(~validCols) = 0; % 无效列权重置零

weights = weights / sum(weights); % 最终归一化


%% 数据准备与验证

weights = [0.32, 0.31, 0.27]; % 输入权重

labels = {'平均寿命', '慢性发病率', '健康满意度'};


% 验证权重总和

total = sum(weights);

if abs(total - 1) > 1e-6

    fprintf('警告：权重总和为%.2f，自动补充"其他"类别\n', total);

    weights = [weights, 1 - total];

    labels = [labels, {'其他'}];

end

```

```

%% 可视化定制

figure('Color','white','Position',[100 100 800 600])

% 创建饼状图

h = pie(weights, labels);

% 设置颜色方案 (MATLAB 默认色系加强版)

colors = lines(length(weights));

for i = 1:length(h)/2

    set(h(2*i-1), 'FaceColor', colors(i,:), 'EdgeColor','w', 'LineWidth',2);

end

% 添加百分比标签

textObjs = findobj(h, 'Type', 'text');

percentValues = get(textObjs, 'String');

for i = 1:length(textObjs)

    textObjs(i).FontSize = 14;

    textObjs(i).FontWeight = 'bold';

end

% 添加图例和标题

legend(labels, 'Location', 'eastoutside', 'FontSize',12)

```

```

title('健康评估指标权重分布', 'FontSize',16, 'FontWeight','bold')

% 设置背景和坐标轴

ax = gca;

ax.FontSize = 12;

ax.Box = 'off';

%% 数据准备

regions = {'贺兰县', '金凤区', '灵武市', '西夏区', '永宁县', '兴庆区'};

parkCounts = [23, 31, 15, 18, 10, 43];

%% 可视化设置

figure('Position', [100 100 900 500]) % 设置图形窗口大小

barHandle = bar(parkCounts, 'FaceColor', [0.2 0.6 0.8], 'BarWidth', 0.6); %

创建蓝色渐变柱状图

%% 坐标轴和标签美化

ax = gca;

% 设置横坐标标签

set(ax, 'XTick', 1:length(regions),...

    'XTickLabel', regions,...

    'XTickLabelRotation', 25,...

```

```

        'FontSize', 12)

% 设置纵坐标范围
ylim([0 max(parkCounts)*1.15])

% 添加坐标轴标题
ylabel('公园数量 (个)', 'FontSize', 14, 'FontWeight','bold')
title('银川市各区县公园数量分布', 'FontSize', 16, 'FontWeight','bold')

%% 添加数据标签
text(1:length(parkCounts),...
     parkCounts + max(parkCounts)*0.02,...
     num2str(parkCounts'),...
     'HorizontalAlignment', 'center',...
     'VerticalAlignment', 'bottom',...
     'FontSize', 12,...
     'Color', [0.3 0.3 0.3])

%% 辅助元素添加
grid on
box off

```

```
%% 突出显示最大值
```

```
maxIndex = find(parkCounts == max(parkCounts));
```

```
barHandle.FaceColor = 'flat';
```

```
barHandle.CData(maxIndex,:) = [0.8 0.2 0.2]; % 将最大值设为红色
```

```
%% 添加注释
```

```
annotation('textarrow', [0.8 0.75], [0.8 0.7],...
```

```
    'String', '兴庆区公园数量最多',...
```

```
    'FontSize', 12,...
```

```
    'HeadWidth', 15,...
```

```
    'LineWidth', 1.5)
```

```
%% 数据准备
```

```
regions = {'贺兰县', '金凤区', '灵武市', '西夏区', '永宁县', '兴庆区'};
```

```
parkCounts = [23, 31, 15, 18, 10, 43];
```

```
%% 可视化设置
```

```
figure('Position', [100 100 900 500]) % 设置图形窗口大小
```

```
barHandle = bar(parkCounts, 'FaceColor', [0.2 0.6 0.8], 'BarWidth', 0.6); %
```

```
创建蓝色渐变柱状图
```

```
%% 坐标轴和标签美化
```

```
ax = gca;
```

```
% 设置横坐标标签
```

```
set(ax, 'XTick', 1:length(regions),...
```

```
    'XTickLabel', regions,...
```

```
    'XTickLabelRotation', 25,...
```

```
    'FontSize', 12)
```

```
% 设置纵坐标范围
```

```
ylim([0 max(parkCounts)*1.15])
```

```
% 添加坐标轴标题
```

```
ylabel('公园数量 (个)', 'FontSize', 14, 'FontWeight','bold')
```

```
title('银川市各区县公园数量分布', 'FontSize', 16, 'FontWeight','bold')
```

```
%% 添加数据标签
```

```
text(1:length(parkCounts),...
```

```
    parkCounts + max(parkCounts)*0.02,...
```

```
    num2str(parkCounts'),...
```

```
    'HorizontalAlignment', 'center',...
```

```
    'VerticalAlignment', 'bottom',...
```

```
    'FontSize', 12,...
```



```

        'Color', [0.3 0.3 0.3])

%% 辅助元素添加

grid on

box off

%% 突出显示最大值

maxIndex = find(parkCounts == max(parkCounts));

barHandle.FaceColor = 'flat';

barHandle.CData(maxIndex,:) = [0.8 0.2 0.2]; % 将最大值设为红色

%% 添加注释

annotation('textarrow', [0.8 0.75], [0.8 0.7],...

    'String', '兴庆区公园数量最多',...

    'FontSize', 12,...

    'HeadWidth', 15,...

    'LineWidth', 1.5)

function normalizedData = normalizePositiveIndicators(data,

positiveCols)

% 初始化输出矩阵

```

```

normalizedData = data;

% 遍历所有正向指标列
for col = positiveCols
    % 提取当前列数据
    currentCol = data(:, col);

    % 计算最小值和范围
    minVal = min(currentCol);
    maxVal = max(currentCol);
    range = maxVal - minVal;

    % 避免除零错误
    if range == 0
        warning('列 %d 所有值相同, 无法标准化! 已保持原始值', col);
        continue;
    end

    % 执行标准化:  $x^* = (max - x) / (max - min)$ 
    normalizedData(:, col) = (maxVal - currentCol) / range;
end
end

```

```
%% 示例数据测试
```

```
rawData = [ 82.5  
            86.4  
            83.7  
            87.6  
            89.2  
            85  
            ];
```

```
positiveColumns = [1]; % 示例中单列数据
```

```
% 执行标准化
```

```
normalizedData = normalizePositiveIndicators(rawData,  
positiveColumns);
```

```
1
```

```
%% 结果显示
```

```
disp('==== 原始数据 =====');
```

```
disp(rawData);
```

```
disp('==== 标准化结果 =====');
```

```
disp(normalizedData);
```

```
disp('==== 验证计算 =====');
```

```
fprintf('兴庆区计算值: %.2f \n', normalizedData(1));  
fprintf('西夏区计算值: %.2f \n', normalizedData(2));  
fprintf('贺兰区计算值: %.2f \n', normalizedData(3));  
fprintf('永宁县计算值: %.2f \n', normalizedData(4));  
fprintf('金凤区计算值: %.2f \n', normalizedData(5));  
fprintf('灵武市计算值: %.2f \n', normalizedData(6));
```

```
function normalizedData = normalizePositiveIndicators(data,  
positiveCols)
```

```
    % 初始化输出矩阵
```

```
    normalizedData = data;
```

```
    % 遍历所有正向指标列
```

```
    for col = positiveCols
```

```
        % 提取当前列数据
```

```
        currentCol = data(:, col);
```

```
        % 计算最小值和范围
```

```
        minVal = min(currentCol);
```

```
        maxVal = max(currentCol);
```

```
        range = maxVal - minVal;
```

```

% 避免除零错误

if range == 0

    warning('列 %d 所有值相同, 无法标准化! 已保持原始值', col);

    continue;

end

% 执行标准化:  $x^* = (\max - x) / (\max - \min)$ 

normalizedData(:, col) = (maxVal - currentCol) / range;

end

end

%% 示例数据测试

rawData = [15
           31
           10
           23
           18
           43];

positiveColumns = [1]; % 示例中单列数据

```

```
% 执行标准化

normalizedData = normalizePositiveIndicators(rawData,
positiveColumns);
```

```
%% 结果显示

disp('==== 原始数据 =====');

disp(rawData);

disp('==== 标准化结果 =====');

disp(normalizedData);

disp('==== 验证计算 =====');

fprintf('兴庆区计算值: %.2f \n', normalizedData(1));
fprintf('西夏区计算值: %.2f \n', normalizedData(2));
fprintf('贺兰区计算值: %.2f \n', normalizedData(3));
fprintf('永宁县计算值: %.2f \n', normalizedData(4));
fprintf('金凤区计算值: %.2f \n', normalizedData(5));
fprintf('灵武市计算值: %.2f \n', normalizedData(6));
```

```
function normalizedData = normalizePositiveIndicators(data,
positiveCols)
```

```
    % 初始化输出矩阵

    normalizedData = data;
```

```

% 遍历所有正向指标列

for col = positiveCols

    % 提取当前列数据

    currentCol = data(:, col);

    % 计算最小值和范围

    minVal = min(currentCol);

    maxVal = max(currentCol);

    range = maxVal - minVal;

    % 避免除零错误

    if range == 0

        warning('列 %d 所有值相同, 无法标准化! 已保持原始值', col);

        continue;

    end

    % 执行标准化:  $x^* = (max - x) / (max - min)$ 

    normalizedData(:, col) = (maxVal - currentCol) / range;

end

end

```

```
%% 示例数据测试
```

```
rawData = [7  
            18  
            2  
            3  
            8  
            12];
```

```
positiveColumns = [1]; % 示例中单列数据
```

```
% 执行标准化
```

```
normalizedData = normalizePositiveIndicators(rawData,  
positiveColumns);
```

```
%% 结果显示
```

```
disp('==== 原始数据 =====');
```

```
disp(rawData);
```

```
disp('==== 标准化结果 =====');
```

```
disp(normalizedData);
```

```
disp('==== 验证计算 =====');
```

```
fprintf('兴庆区计算值: %.2f \n', normalizedData(1));
```

```
fprintf('西夏区计算值: %.2f \n', normalizedData(2));
```



```
fprintf('贺兰区计算值: %.2f \n', normalizedData(3));
```

```
fprintf('永宁县计算值: %.2f \n', normalizedData(4));
```

```
fprintf('金凤区计算值: %.2f \n', normalizedData(5));
```

```
fprintf('灵武市计算值: %.2f \n', normalizedData(6));
```

```
%% 康养城市多目标优化系统（脚本版）
```

```
% 功能：基于 NSGA-II 算法优化设施布局，平衡覆盖率、公平性、可达性和成本
```

```
clearvars; close all; clc;
```

```
rng(2023); % 固定随机种子保证可复现性
```

```
%% ===== 1.模拟数据生成
```

```
=====
```

```
)
```

```
n = 5;
```

```
latitudes = 31.2 + 0.03 * randn(n, 1); % 自动生成 n 行纬度数据
```

```
longitudes = 121.4 + 0.02 * randn(n, 1); % 自动生成 n 行经度数据
```

```
capacities = randi([100, 300], n, 1); % 自动生成 n 行容量数据
```

```
unitCosts = randi([500, 1500], n, 1); % 自动生成 n 行单位成本数据
```

```
facilityData = table(latitudes, longitudes, capacities, unitCosts, ...
```

```
    'VariableNames', {'Latitude', 'Longitude', 'Capacity', 'UnitCost'});
```

```

% 现有设施地理坐标转换（生成 X/Y 列）

projectCoord = @(lat, lon) [lon * 1e4 - 1214737, lat * 1e4 - 312304];

coordMatrix = projectCoord(facilityData.Latitude,
facilityData.Longitude);

facilityData.X = coordMatrix(:, 1);

facilityData.Y = coordMatrix(:, 2);


% 1.2 人口统计数据（表格列索引）

% 自动生成相同行数的人口数据（与设施数据行数一致）

popSize = size(facilityData, 1);

popLatitudes = 31.2 + 0.03 * randn(popSize, 1); % 自动生成 popSize 行
纬度数据

popLongitudes = 121.4 + 0.02 * randn(popSize, 1); % 自动生成 popSize
行经度数据

popCounts = randi([2000, 6000], popSize, 1); % 自动生成 popSize 行人口
数数据

populationData = table(popLatitudes, popLongitudes, popCounts, ...
    'VariableNames', {'Latitude', 'Longitude', 'Population'});


% 人口数据地理坐标转换（生成 X/Y 列）

coordMatrix = projectCoord(populationData.Latitude,
populationData.Longitude);

```

```

populationData.X = coordMatrix(:, 1);

populationData.Y = coordMatrix(:, 2);


% 1.3 路网数据生成

% 优化：将赋值拆分为变量，提高可读性

roadNodes = [121.4737 31.2304; 121.4891 31.2123; 121.5050 31.2401;
121.4755 31.2255];

roadEdges = [1 2; 2 3; 1 4];

% 定义一个函数来检查边数据与其他数据的数量一致性

function checkEdgeDataConsistency(edges, speeds, lengths)

    assert(size(edges, 1) == size(speeds, 1), '边的数量与道路速度数据的数
量不一致');

    assert(size(edges, 1) == size(lengths, 1), '边的数量与道路长度数据的数
量不一致');

end


% 先定义道路速度和长度数据

roadSpeeds = [40; 50; 60];

roadLengths = [1500; 2000; 1000];


% 调用函数进行检查

checkEdgeDataConsistency(roadEdges, roadSpeeds, roadLengths);

```

```

roadNetwork = struct();

roadNetwork.Nodes = roadNodes;

roadNetwork.Edges = roadEdges;

roadNetwork.Speed = roadSpeeds;

roadNetwork.Length = roadLengths;

roadNetwork.Graph = graph(roadNetwork.Edges(:,1),
roadNetwork.Edges(:,2), ...

    roadNetwork.Length ./ roadNetwork.Speed * 0.06); % 时间成本 (分钟)

% 1.4 候选设施数据 (表格行索引)

% 自动生成相同行数的候选设施数据 (示例生成 3 行)

candSize = 3;

candLatitude = 31.2 + 0.03 * randn(candSize, 1); % 自动生成 candSize
行纬度数据

candLongitude = 121.4 + 0.02 * randn(candSize, 1); % 自动生成
candSize 行经度数据

candCost = randi([3000, 7000], candSize, 1); % 自动生成 candSize 行成本
数据

candidateSites = table(candLatitude, candLongitude, candCost, ...

    'VariableNames', {'Latitude', 'Longitude', 'Cost'});

% 候选设施地理坐标转换 (生成 X/Y 列)

```

```

coordMatrix = projectCoord(candidateSites.Latitude,
candidateSites.Longitude);

candidateSites.X = coordMatrix(:, 1);
candidateSites.Y = coordMatrix(:, 2);


% 基础校验

assert(~isempty(candidateSites), '错误：候选设施数据 candidateSites 未正
确生成');

numNew = height(candidateSites);
numExist = height(facilityData);

disp(['验证 numNew 定义：当前候选设施数为', num2str(numNew), ',
numNew 值为', num2str(numNew)]);


%% ===== 2.优化模型配置
=====

nvars = numNew + numExist + (numNew + numExist); % 决策变量总数

% 优化：将赋值拆分为变量，提高可读性

lbNew = zeros(1, numNew);

lbExist = zeros(1, numExist);

lbRadius = 500 * ones(1, numNew + numExist);

lb = [lbNew, lbExist, lbRadius];

```

```

ubNew = ones(1, numNew);

ubExist = ones(1, numExist);

ubRadius = 4000 * ones(1, numNew + numExist);

ub = [ubNew, ubExist, ubRadius];


% 优化：将选项参数拆分为变量，提高可读性和可维护性

popSize = 100;

paretoFrac = 0.3;

crossoverFrac = 0.8;

maxGens = 50;

displayOpt = 'iter';

useParallelOpt = false;


options = optimoptions('gamultiobj', ...
    'PopulationSize', popSize, ...
    'ParetoFraction', paretoFrac, ...
    'CrossoverFraction', crossoverFrac, ...
    'MaxGenerations', maxGens, ...
    'Display', displayOpt, ...
    'UseParallel', useParallelOpt);

%% ===== 3.运行优化

=====

```

```

% 定义重试次数，当优化失败时尝试重新运行优化过程

maxRetries = 3;

retryCount = 0;

success = false;


% 定义一个结构体来保存外部变量，以便在局部函数中使用

externalVars = struct('numNew', numNew, 'numExist', numExist,
'candidateSites', candidateSites, 'facilityData', facilityData,
'populationData', populationData, 'roadNetwork', roadNetwork, 'nvars',
nvars, 'lb', lb, 'ub', ub, 'options', options, 'retryCount', retryCount,
'success', success, 'maxRetries', maxRetries);


% 定义局部函数的封装函数，用于将局部函数作为嵌套函数使用

function main(externalVars)

    retryCount = externalVars.retryCount;

    success = externalVars.success;

    maxRetries = externalVars.maxRetries;

    while ~success && retryCount < maxRetries

        try

            % 通过匿名函数传递外部变量给局部函数 moObjective

            [~, optObj] = gamultiobj(@(x) moObjective(x,
externalVars.numNew, externalVars.numExist,

```

```

externalVars.candidateSites, externalVars.facilityData,
externalVars.populationData, externalVars.roadNetwork), ...
        externalVars.nvars, [], [], [], [], externalVars.lb,
externalVars.ub, [], externalVars.options);

disp('优化成功完成! ');

plotOptimizationResults(optObj);

success = true;

catch ME

    retryCount = retryCount + 1;

    if retryCount < maxRetries

        disp(['优化尝试 ', num2str(retryCount), ' 失败, 准备进行
第 ', num2str(retryCount + 1), ' 次尝试...']);

        disp(['错误信息: ', ME.message]);

    else

        disp('优化多次尝试均失败, 请检查参数设置或数据! ');

        disp(['最终错误信息: ', ME.message]);

    end

end

end

end

% 局部函数定义 (嵌套函数)

function [uncovered, gini, avgTime, cost] = moObjective(x, numNew,

```



```

numExist, candidateSites, facilityData, populationData, roadNetwork)

newSites = x(1:numNew);

expandRatio = x(numNew + 1:numNew + numExist);

serviceRadius = x(numNew + numExist + 1:end);


selectedSites = candidateSites(newSites == 1, :);


% 空值处理

if isempty(selectedSites)

    % 创建与 facilityData 列数匹配的空表 (6 列)

    newFacilities = table('Size', [0 6], 'VariableNames', {'Latitude',
'Longitude', 'Capacity', 'UnitCost', 'X', 'Y'});

else

    newFacilities = table(...

        selectedSites.Latitude, ...

        selectedSites.Longitude, ...

        zeros(height(selectedSites), 1), ...

        zeros(height(selectedSites), 1), ...

        selectedSites.X, ...

        selectedSites.Y, ...

        'VariableNames', {'Latitude', 'Longitude', 'Capacity',
'UnitCost', 'X', 'Y'});

```

```

end

allFacilities = [facilityData; newFacilities];

% 确保 expandRatio 长度与现有设施数匹配
if length(expandRatio) < height(facilityData)
    expandRatio = [expandRatio, zeros(1, height(facilityData) -
length(expandRatio))];
end

allFacilities.Capacity(1:height(facilityData)) =
allFacilities.Capacity(1:height(facilityData)) .* (1 + expandRatio);

% 校验 serviceRadius 长度与设施数匹配
if length(serviceRadius) < height(allFacilities)
    % 扩展 serviceRadius 到所需长度，使用最后一个元素填充
    serviceRadius = [serviceRadius, repmat(serviceRadius(end),
1, height(allFacilities) - length(serviceRadius))];
elseif length(serviceRadius) > height(allFacilities)
    % 截断 serviceRadius 到所需长度
    serviceRadius = serviceRadius(1:height(allFacilities));
end

[coverage, ~] = calcCoverage(allFacilities, serviceRadius,

```

```

populationData);

    uncovered = 1 - coverage;

    [supply, demand] = calcSupplyDemand(allFacilities,
populationData);

    gini = calcGini(supply, demand);

    accessTime = calcAccessTime(allFacilities, roadNetwork,
populationData);

    avgTime = mean(accessTime);

    cost = sum(newSites .* candidateSites.Cost) +
sum(expandRatio .* facilityData.UnitCost .* facilityData.Capacity);
end

function [coverage, distMat] = calcCoverage(facilities, radius,
populationData)

    facXY = [facilities.X, facilities.Y];

    popXY = [populationData.X, populationData.Y];

    [idx, dist] = knnsearch(facXY, popXY); % idx 范围
1~height(facilities)

```

```

% 限制索引不超过 radius 长度, 避免越界

idx = min(idx, length(radius));

covered = dist <= radius(idx);

coverage = sum(populationData.Population(covered)) /
sum(populationData.Population);

distMat = dist;

end

```

```

function gini = calcGini(supply, demand)

% 检查分母是否为零, 避免除零错误

if sum(supply) == 0 || sum(demand) == 0

    gini = 1; % 极端不公平情况

    return;

end

ratio = demand ./ supply;

[~, idx] = sort(ratio);

cumDemand = cumsum(demand(idx)) / sum(demand);

cumSupply = cumsum(supply(idx)) / sum(supply);

A = trapz(cumDemand, cumSupply);

gini = 1 - 2 * A;

end

```

```

function accessTime = calcAccessTime(facilities, roadNetwork,
populationData)

    accessTime = zeros(height(populationData), 1);

    for i = 1:height(populationData)

        popNode = find(ismember(roadNetwork.Nodes,
[populationData.Longitude(i), populationData.Latitude(i)], 'rows'), 1);

        if isempty(popNode)

            accessTime(i) = Inf;

            continue;

        end

        minTime = Inf;

        for j = 1:height(facilities)

            facNode = find(ismember(roadNetwork.Nodes,
[facilities.Longitude(j), facilities.Latitude(j)], 'rows'), 1);

            if isempty(facNode)

                continue;

            end

            % 检查图中是否存在路径

            if ispath(roadNetwork.Graph, popNode, facNode)

                [~, pathTime] = shortestpath(roadNetwork.Graph,

```

```

popNode, facNode);

        if pathTime < minTime
            minTime = pathTime;
        end
    end
end

    endTime(i) = minTime;
end

end

function [supply, demand] = calcSupplyDemand(facilities,
populationData)

    popSize = height(populationData);

    facSize = height(facilities);

    % 修正矩阵维度问题

    dist = sqrt(sum((repmat([populationData.X, populationData.Y],
1, facSize) - ...

        repmat([facilities.X, facilities.Y].', popSize, 1)).^2, 2));

    [~, idx] = min(dist, [], 2);

    supply = facilities.Capacity(idx);

    demand = populationData.Population;

end

```

```

function plotOptimizationResults(optObj)

    % 检查 optObj 是否为空

    if isempty(optObj)

        disp('错误：优化结果为空，无法绘制图形');

        return;

    end

    % 目标函数说明：optObj 列分别为 [uncovered, gini, avgTime,
cost]

    figure('Position', [100 100 1000 600]);

    % 子图 1：覆盖率 vs 公平性（基尼系数）

    subplot(2,2,1);

    scatter(optObj(:,1), optObj(:,2), 30, [0 0.5 0.8], 'filled');

    xlabel('未覆盖率 (Uncovered Ratio)');

    ylabel('公平性（基尼系数） (Gini Index)');

    title('未覆盖率与公平性权衡');

    grid on;

    % 子图 2：覆盖率 vs 平均访问时间

    subplot(2,2,2);

    scatter(optObj(:,1), optObj(:,3), 30, [0.8 0.2 0.3], 'filled');

```

```

xlabel('未覆盖率 (Uncovered Ratio)');

ylabel('平均访问时间 (min) (Avg Access Time)');

title('未覆盖率与可达性权衡');

grid on;

% 子图 3: 公平性 vs 总成本

subplot(2,2,3);

scatter(optObj(:,2), optObj(:,4), 30, [0.3 0.8 0.4], 'filled');

xlabel('公平性 (基尼系数) (Gini Index)');

ylabel('总成本 (Cost)');

title('公平性与成本权衡');

grid on;

% 子图 4: 平行坐标图展示所有目标

subplot(2,2,4);

parallelcoords(optObj, 'Colormap', jet(size(optObj,1)));

set(gca, 'XtickLabel', {'未覆盖率','基尼系数','平均时间','总成本'},

'FontSize', 8);

ylabel('目标函数值标准化后');

title('多目标帕累托前沿平行坐标图');

end

end

```



```

% 调用封装函数开始执行

main(externalVars);

%% 康养城市多目标优化系统（脚本版）

% 功能：基于 NSGA-II 算法优化设施布局，平衡覆盖率、公平性、可达性和成本

clearvars; close all; clc;

rng(2023); % 固定随机种子保证可复现性

%% ===== 1.模拟数据生成
=====

% 1.1 现有康养设施数据

facilityData = table(...

    ['Existing_1'; 'Existing_2'], ...

    [31.2304; 31.2123], ...

    [121.4737; 121.4891], ...

    [150; 200], ...

    [800; 1200], ...

    'VariableNames', {'FacilityID', 'Latitude', 'Longitude', 'Capacity',

'UnitCost'});

```

```

% 现有设施地理坐标转换 (生成 X/Y 列)

projectCoord = @(lat, lon) [lon * 1e4 - 1214737, lat * 1e4 - 312304];

coordMatrix = projectCoord(facilityData.Latitude,
facilityData.Longitude);

facilityData.X = coordMatrix(:, 1);

facilityData.Y = coordMatrix(:, 2);


% 1.2 人口统计数据 (表格列索引)

populationData = table(...

    [31.2350; 31.2200; 31.2401], ... % 行索引: 1-4 行 (隐式)

    [121.4800; 121.4900; 121.5050], ...

    [5000; 4000; 4500], ...

    'VariableNames', {'Latitude', 'Longitude', 'Population'});


% 人口数据地理坐标转换 (生成 X/Y 列)

coordMatrix = projectCoord(populationData.Latitude,
populationData.Longitude);

populationData.X = coordMatrix(:, 1);

populationData.Y = coordMatrix(:, 2);


% 1.3 路网数据生成

roadNetwork = struct();

```

```

roadNetwork.Nodes = [121.4737 31.2304; 121.4891 31.2123;
121.5050 31.2401; 121.4755 31.2255];

roadNetwork.Edges = [1 2; 2 3; 1 4];

roadNetwork.Speed = [40; 50; 60];

roadNetwork.Length = [1500; 2000; 1000];

roadNetwork.Graph = graph(roadNetwork.Edges(:,1),
roadNetwork.Edges(:,2), ...

    roadNetwork.Length ./ roadNetwork.Speed * 0.06); % 时间成本
(分钟)

```

% 1.4 候选设施数据（表格行索引）

```

candidateSites = table(...

    'Candidate_1', ... % 新增候选设施 ID 列（列索引 1）

    31.2250, ... % 原 2 行，减半后保留 1 行

    121.4750, ... % 原 2 行，减半后保留 1 行

    5000, ... % 原 2 行，减半后保留 1 行

    'VariableNames', {'FacilityID', 'Latitude', 'Longitude', 'Cost'});

% 候选设施地理坐标转换（生成 X/Y 列）

coordMatrix = projectCoord(candidateSites.Latitude,
candidateSites.Longitude);

candidateSites.X = coordMatrix(:, 1);

```

```

candidateSites.Y = coordMatrix(:, 2);

% 基础校验

assert(~isempty(candidateSites), '错误：候选设施数据 candidateSites
未正确生成');

numNew = height(candidateSites);

numExist = height(facilityData);

disp(['验证 numNew 定义：当前候选设施数为', num2str(numNew), ',
numNew 值为', num2str(numNew)]);

%% ===== 2.优化模型配置
=====

nvars = numNew + numExist + (numNew + numExist); % 决策变量
总数

lb = [zeros(1, numNew), zeros(1, numExist), 500 * ones(1, numNew +
numExist)];

ub = [ones(1, numNew), ones(1, numExist), 4000 * ones(1, numNew
+ numExist)];

options = optimoptions('gamultiobj', ...

    'PopulationSize', 100, ...

    'ParetoFraction', 0.3, ...

```

```
'CrossoverFraction', 0.8, ...
```

```
'MaxGenerations', 50, ...
```

```
'Display', 'iter', ...
```

```
'UseParallel', false);
```

```
%% ===== 3.运行优化
```

```
=====
```

```
try
```

```
% 通过匿名函数传递外部变量给局部函数 moObjective
```

```
[~, optObj] = gamultiobj(@(x) moObjective(x, numNew,  
numExist, candidateSites, facilityData, populationData, roadNetwork), ...  
nvars, [], [], [], lb, ub, [], options);
```

```
disp('优化成功完成! ');
```

```
plotOptimizationResults(optObj);
```

```
catch ME
```

```
disp('Error in optimization:');
```

```
disp(ME.message);
```

```
end
```

```
%% 局部函数定义（脚本末尾）
```

```
function [uncovered, gini, avgTime, cost] = moObjective(x, numNew,  
numExist, candidateSites, facilityData, populationData, roadNetwork)
```

```

newSites = x(1:numNew);

expandRatio = x(numNew + 1:numNew + numExist);

serviceRadius = x(numNew + numExist + 1:end);


% 验证逻辑索引范围

assert(length(newSites) == numNew, 'newSites 长度与候选设施数
numNew 不匹配')

validIndices = (newSites == 1) & (1:length(newSites) <=
numNew); % 过滤超出范围的 true 值

selectedSites = candidateSites(validIndices, :);


% 空值处理

if isempty(selectedSites)

    % 创建与 facilityData 列数匹配的空表 (6 列)

    newFacilities = table();

    []; ... % FacilityID 空数组

    []; ... % Latitude 空数组

    []; ... % Longitude 空数组

    []; ... % Capacity 空数组

    []; ... % UnitCost 空数组

    []; ... % X 空数组

    []; ... % Y 空数组

```

```

        'VariableNames', {'FacilityID', 'Latitude', 'Longitude',
'Capacity', 'UnitCost', 'X', 'Y'};

    else

        newFacilities = table(...

            ['Candidate_' num2str(1:height(selectedSites))], ...    %
新增设施 ID 列

            selectedSites.Latitude, ...

            selectedSites.Longitude, ...

            zeros(height(selectedSites), 1), ...

            zeros(height(selectedSites), 1), ...

            selectedSites.X, ...

            selectedSites.Y, ...

            'VariableNames', {'FacilityID', 'Latitude', 'Longitude',
'Capacity', 'UnitCost', 'X', 'Y'});

    end

    allFacilities = [facilityData; newFacilities];

    allFacilities.Capacity = allFacilities.Capacity .* (1 + expandRatio);

    % 校验 serviceRadius 长度与设施数匹配

    assert(length(serviceRadius) >= height(allFacilities), ...

        ['serviceRadius 长度不足（当前',

```

```
num2str(length(serviceRadius)), ', 需要至少', num2str(height(allFacilities)),  
) ']);
```

```
[coverage, ~] = calcCoverage(allFacilities, serviceRadius,  
populationData);
```

```
uncovered = 1 - coverage;
```

```
[supply, demand] = calcSupplyDemand(allFacilities,  
populationData);
```

```
gini = calcGini(supply, demand);
```

```
accessTime = calcAccessTime(allFacilities, roadNetwork,  
populationData);
```

```
avgTime = mean(accessTime);
```

```
cost = sum(newSites .* candidateSites.Cost) +  
sum(expandRatio .* facilityData.UnitCost .* facilityData.Capacity);  
end
```

```
function [coverage, distMat] = calcCoverage(facilities, radius,  
populationData)
```

```
facXY = [facilities.X, facilities.Y];
```



```

popXY = [populationData.X, populationData.Y];

[idx, dist] = knnsearch(facXY, popXY); % idx 范围
1~height(facilities)

% 限制索引不超过 radius 长度, 避免越界
idx = min(idx, length(radius));

covered = dist <= radius(idx);

coverage = sum(populationData.Population(covered)) /
sum(populationData.Population);

distMat = dist;

end

```

```

function gini = calcGini(supply, demand)

ratio = demand ./ supply;

[~, idx] = sort(ratio);

cumDemand = cumsum(demand(idx)) / sum(demand);

cumSupply = cumsum(supply(idx)) / sum(supply);

A = trapz(cumDemand, cumSupply);

gini = 1 - 2 * A;

end

```

```

function accessTime = calcAccessTime(facilities, roadNetwork,
populationData)

    accessTime = zeros(height(populationData), 1);

    for i = 1:height(populationData)

        popNode = find(ismember(roadNetwork.Nodes,
[populationData.Longitude(i), populationData.Latitude(i)], 'rows'), 1);

        if isempty(popNode)

            accessTime(i) = Inf;

            continue;

        end

        minTime = Inf;

        for j = 1:height(facilities)

            facNode = find(ismember(roadNetwork.Nodes,
[facilities.Longitude(j), facilities.Latitude(j)], 'rows'), 1);

            if isempty(facNode)

                continue;

            end

            [~, pathTime] = shortestpath(roadNetwork.Graph,
popNode, facNode);

            if pathTime < minTime

                minTime = pathTime;

```

```

        end
    end
    accessTime(i) = minTime;
end
end

```

```

function [supply, demand] = calcSupplyDemand(facilities,
populationData)

    popSize = height(populationData);
    facSize = height(facilities);
    dist = sqrt(sum((repmat([populationData.X; populationData.Y],
1, facSize) - ...
        repmat([facilities.X; facilities.Y].', popSize, 1)).^2, 2));
    [~, idx] = min(dist, [], 2);
    supply = facilities.Capacity(idx);
    demand = populationData.Population;
end

```

```

function plotOptimizationResults(optObj)

    % 目标函数说明: optObj 列分别为 [uncovered, gini, avgTime,
cost]

    figure('Position', [100 100 1000 600]);

```

```
% 子图 1: 覆盖率 vs 公平性 (基尼系数)

subplot(2,2,1);

scatter(optObj(:,1), optObj(:,2), 30, [0 0.5 0.8], 'filled');

xlabel('未覆盖率 (Uncovered Ratio)');

ylabel('公平性 (基尼系数) (Gini Index)');

title('未覆盖率与公平性权衡');

grid on;
```

```
% 子图 2: 覆盖率 vs 平均访问时间

subplot(2,2,2);

scatter(optObj(:,1), optObj(:,3), 30, [0.8 0.2 0.3], 'filled');

xlabel('未覆盖率 (Uncovered Ratio)');

ylabel('平均访问时间 (min) (Avg Access Time)');

title('未覆盖率与可达性权衡');

grid on;
```

```
% 子图 3: 公平性 vs 总成本

subplot(2,2,3);

scatter(optObj(:,2), optObj(:,4), 30, [0.3 0.8 0.4], 'filled');

xlabel('公平性 (基尼系数) (Gini Index)');

ylabel('总成本 (Cost)');
```

```

title('公平性与成本权衡');

grid on;

% 子图 4：平行坐标图展示所有目标

subplot(2,2,4);

parallelcoords(optObj, 'Colormap', jet(size(optObj,1)));

set(gca, 'XtickLabel', {'未覆盖率','基尼系数','平均时间','总成本'},

'FontSize', 8);

ylabel('目标函数值标准化后');

title('多目标帕累托前沿平行坐标图');

end

%% 数据准备与验证

weights = [0.3184,0.2277,0.1721,0.2818]; % 输入权重

labels = {'康养城市资源','环境质量','居民健康状况','经济发展'};

% 验证权重总和

total = sum(weights);

if abs(total - 1) > 1e-6

    fprintf('警告：权重总和为%.2f，自动补充"其他"类别\n', total);

    weights = [weights, 1 - total];

    labels = [labels, {'其他'}];

```

```

end

%% 可视化定制

figure('Color','white','Position',[100 100 800 600])

% 创建饼状图

h = pie(weights, labels);

% 设置颜色方案 (MATLAB 默认色系加强版)

colors = lines(length(weights));

for i = 1:length(h)/2

    set(h(2*i-1), 'FaceColor', colors(i,:), 'EdgeColor','w', 'LineWidth',2);

end

% 添加百分比标签

textObjs = findobj(h, 'Type', 'text');

percentValues = get(textObjs, 'String');

for i = 1:length(textObjs)

    textObjs(i).FontSize = 14;

    textObjs(i).FontWeight = 'bold';

end

```

```

% 添加图例和标题

legend(labels, 'Location', 'eastoutside', 'FontSize',12)

title('经济发展权重', 'FontSize',16, 'FontWeight','bold')


% 设置背景和坐标轴

ax = gca;

ax.FontSize = 12;

ax.Box = 'off';


%% MATLAB 负向指标标准化计算函数

% 功能：对数据矩阵中指定的负向指标列进行标准化处理

% 输入：

%   data - 原始数据矩阵 (n×m, n 样本数, m 指标数)

%   negativeCols - 负向指标列索引 (如[1,2]表示前两列为负向指标)

% 输出：

%   normalizedData - 标准化后的数据矩阵


function normalizedData = normalizeNegativeIndicators(data,
negativeCols)

    % 初始化输出矩阵

    normalizedData = data;

```

```

% 遍历所有负向指标列

for col = negativeCols

    % 提取当前列数据

    currentCol = data(:, col);

    % 计算最小值和范围

    minVal = min(currentCol);

    maxVal = max(currentCol);

    range = maxVal - minVal;

    % 避免除零错误

    if range == 0

        warning('列 %d 所有值相同, 无法标准化! 已保持原始值', col);

        continue;

    end

    % 执行标准化:  $x^* = (x - \min) / (\max - \min)$ 

    normalizedData(:, col) = (currentCol - minVal) / range;

end

end

%% 示例数据测试

```



```

% 原始数据矩阵 (3 样本×3 指标)

% 列 1: 慢性病发病率 (负向)

% 列 2: 老年人口比例 (负向)

% 列 3: 健康满意度 (正向, 此处不处理)

rawData = [18.2, 22.1, 3.8;
           15.6, 19.4, 4.2;
           23.5, 25.7, 3.2];

% 指定负向指标列 (前两列)

negativeColumns = [1, 2];

% 执行标准化

normalizedData = normalizeNegativeIndicators(rawData,
negativeColumns);

%% 结果显示

disp('==== 原始数据 =====');

disp(rawData);

disp('==== 标准化结果 =====');

disp(normalizedData);

disp('==== 验证计算 =====');

fprintf('慢性病发病率标准化值应等于: (原始值 - 15.6)/(23.5-15.6)\n');

```

```
fprintf('兴庆区计算值: %.2f (理论值 1.00) \n', normalizedData(1,1));  
fprintf('西夏区计算值: %.2f (理论值 0.00) \n', normalizedData(2,1));  
fprintf('贺兰区计算值: %.2f (理论值 0.00) \n', normalizedData(3,1));  
fprintf('永宁县计算值: %.2f (理论值 0.00) \n', normalizedData(4,1));  
fprintf('金凤区计算值: %.2f (理论值 0.00) \n', normalizedData(5,1));  
fprintf('灵武市计算值: %.2f (理论值 0.00) \n', normalizedData(6,1));
```

```
function normalizedData = normalizePositiveIndicators(data,  
positiveCols)
```

```
    % 初始化输出矩阵
```

```
    normalizedData = data;
```

```
    % 遍历所有正向指标列
```

```
    for col = positiveCols
```

```
        % 提取当前列数据
```

```
        currentCol = data(:, col);
```

```
        % 计算最小值和范围
```

```
        minVal = min(currentCol);
```

```
        maxVal = max(currentCol);
```

```

range = maxVal - minVal;

% 避免除零错误

if range == 0
    warning('列 %d 所有值相同, 无法标准化! 已保持原始值', col);
    continue;
end

% 执行标准化:  $x^* = (max - x) / (max - min)$ 

normalizedData(:, col) = (maxVal - currentCol) / range;

end

end

%% 示例数据测试

% 原始数据矩阵 (3 样本×3 指标)

% 列 3: 健康满意度 (正向指标)

rawData = [3.8;
            4.2;
            3.2
            5.0
            3.3
            3.3];

```

```
positiveColumns = [1]; % 示例中单列数据
```

```
% 执行标准化
```

```
normalizedData = normalizePositiveIndicators(rawData,  
positiveColumns);
```

```
%% 结果显示
```

```
disp('==== 原始数据 ====');
```

```
disp(rawData);
```

```
disp('==== 标准化结果 ====');
```

```
disp(normalizedData);
```

```
disp('==== 验证计算 ====');
```

```
fprintf('健康满意度标准化值应等于: (4.2 - 原始值)/(4.2-3.2)\n');
```

```
fprintf('兴庆区计算值: %.2f (理论值 1.00) \n', normalizedData(1));
```

```
fprintf('西夏区计算值: %.2f (理论值 0.00) \n', normalizedData(2));
```

```
fprintf('贺兰区计算值: %.2f (理论值 0.00) \n', normalizedData(3));
```

```
fprintf('永宁县计算值: %.2f (理论值 0.00) \n', normalizedData(4));
```

```
fprintf('金凤区计算值: %.2f (理论值 0.00) \n', normalizedData(5));
```

```
fprintf('灵武市计算值: %.2f (理论值 0.00) \n', normalizedData(6));
```

```
%% 数据准备
```

```
regions = {'贺兰县', '金凤区', '灵武市', '西夏区', '永宁县', '兴庆区'};
```

```

parkCounts = [23, 31, 15, 18, 10, 43];

%% 可视化设置

figure('Position', [100 100 900 500]) % 设置图形窗口大小

barHandle = bar(parkCounts, 'FaceColor', [0.2 0.6 0.8], 'BarWidth', 0.6); %
创建蓝色渐变柱状图

%% 坐标轴和标签美化

ax = gca;

% 设置横坐标标签

set(ax, 'XTick', 1:length(regions),...

    'XTickLabel', regions,...

    'XTickLabelRotation', 25,...

    'FontSize', 12)

% 设置纵坐标范围

ylim([0 max(parkCounts)*1.15])

% 添加坐标轴标题

ylabel('公园数量 (个)', 'FontSize', 14, 'FontWeight','bold')

title('银川市各区县公园数量分布', 'FontSize', 16, 'FontWeight','bold')

```

```
%% 添加数据标签
```

```
text(1:length(parkCounts),...  
     parkCounts + max(parkCounts)*0.02,...  
     num2str(parkCounts'),...  
     'HorizontalAlignment', 'center',...  
     'VerticalAlignment', 'bottom',...  
     'FontSize', 12,...  
     'Color', [0.3 0.3 0.3])
```

```
%% 辅助元素添加
```

```
grid on
```

```
box off
```

```
%% 突出显示最大值
```

```
maxIndex = find(parkCounts == max(parkCounts));  
barHandle.FaceColor = 'flat';  
barHandle.CData(maxIndex,:) = [0.8 0.2 0.2]; % 将最大值设为红色
```

```
%% 添加注释
```

```
annotation('textarrow', [0.8 0.75], [0.8 0.7],...  
           'String', '兴庆区公园数量最多',...  
           'FontSize', 12,...
```

```
'HeadWidth', 15,...
```

```
'LineWidth', 1.5)
```

```
%% 模块 1：数据加载与预处理
```

```
clearvars; close all; clc;
```

```
fprintf('=== 模块 1：数据预处理 ===\n');
```

```
% 1.1 读取康养设施数据
```

```
facilityData = readtable('facilities.csv');
```

```
fprintf('已加载康养设施数据，包含%d 个设施\n', height(facilityData));
```

```
% 1.2 读取人口统计数据
```

```
populationData = readtable('population.csv');
```

```
fprintf('已加载人口数据，包含%d 个统计单元\n', height(populationData));
```

```
% 1.3 地理坐标转换 (WGS84 转笛卡尔坐标)
```

```
[facilityData.X, facilityData.Y] = ll2cart(facilityData.Latitude,  
facilityData.Longitude);
```

```
[populationData.X, populationData.Y] = ll2cart(populationData.Latitude,  
populationData.Longitude);
```

```
% 1.4 数据标准化
```

```

facilityData.Capacity = normalize(facilityData.Capacity, 'range');

populationData.Density = normalize(populationData.Density, 'range');


%% 模块 2: 空间分布分析

fprintf('\n=== 模块 2: 空间特征分析 ===\n');


% 2.1 核密度估计

[gridX, gridY, density] = ksdensity2d(...
    [facilityData.X; populationData.X],...
    [facilityData.Y; populationData.Y],...
    'Bandwidth', 500, 'GridSize', [100 100]);


% 2.2 Voronoi 服务区划分

voronoiCells = voronoin([facilityData.X, facilityData.Y]);


% 2.3 空间自相关分析

morani = spatialAutocorr(populationData.Density);

fprintf('全局 Moran's I 指数 = %.3f (p=%.4f)\n', morani.I, morani.pValue);


%% 模块 3: 公平性评估

fprintf('\n=== 模块 3: 公平性评估 ===\n');

```


% 3.1 构建供需矩阵

```
supply = facilityData.Capacity;  
  
demand = populationData.Population;  
  
adjMatrix = pdist2([facilityData.X, facilityData.Y],...  
                   [populationData.X, populationData.Y]);
```

% 3.2 计算基尼系数

```
gini = 1 - 2*trapz(cumsum(sort(demand))/sum(demand),...  
                  cumsum(supply(sortIdx))/sum(supply));  
  
fprintf('供需基尼系数 = %.3f\n', gini);
```

% 3.3 区位熵分析

```
locationEntropy = (supply./demand) ./ mean(supply./demand);  
  
disp(table(populationData.Region, locationEntropy,...  
           'VariableNames', {'Region','LocationEntropy'}));
```

%% 模块 4: 可达性分析

```
fprintf('\n=== 模块 4: 可达性分析 ===\n');
```

% 4.1 加载路网数据

```
roadNetwork = loadRoadNetwork('road_network.shp');
```

% 4.2 计算最短路径时间

```
[accessTime, paths] = calculateAccessibility(...  
    [populationData.X, populationData.Y],...  
    [facilityData.X, facilityData.Y],...  
    roadNetwork);
```

% 4.3 可达性分级

```
accessLevel = discretize(accessTime,...  
    [0 15 30 60], 'categorical', {'优','良','差'});
```

% %% 模块 5：三维可视化

```
% fprintf('\n=== 模块 5：三维可视化 ===\n');
```

%

% % 5.1 创建地理场景

```
% figure('Position', [100 100 1440 900], 'Color','w')
```

```
% ax = geoaxes('Basemap','satellite');
```

```
% hold(ax, 'on');
```

%

% % 5.2 绘制康养设施

```
% hFacility = geoscatter(ax, facilityData.Latitude, facilityData.Longitude,...
```

```
%     100, facilityData.Capacity, 'filled',...
```

```
%     'MarkerEdgeColor','k',...
```

```

%     'DisplayName','康养设施');

% colormap(ax, jet)

% cBar = colorbar(ax, 'Ticks',0:0.2:1,...

%     'TickLabels',{'低容量',' ',' ','高容量'});

% cBar.Label.String = '标准化容量';

%

% % 5.3 叠加人口热力图

% hHeatmap = geodensityplot(ax, populationData.Latitude,

populationData.Longitude,...

%     'Radius', 3000, 'FaceColor','interp',...

%     'ColorMap',hot, 'FaceAlpha',0.5);

%

% % 5.4 标注可达性等级

% for i = 1:height(populationData)

%     text(ax, populationData.Latitude(i),

populationData.Longitude(i),...

%         char(accessLevel(i)),...

%         'Color','w', 'FontSize',8, 'HorizontalAlignment','center');

% end

%

% % 5.5 设置图例与标题

% legend([hFacility, hHeatmap], {'康养设施','人口密度'},

```

```
'Location','northeast')  
  
% title(ax, '康养资源三维分布与可达性分析')  
  
% hold(ax, 'off');
```